

第 9 章 动态规划初步

学习目标

- ☑ 理解状态和状态转移方程
- ☑ 理解最优子结构和重叠子问题
- ☑ 熟练运用递推法和记忆化搜索求解数字三角形问题
- ☑ 熟悉 DAG 上动态规划的常见思路、两种状态定义方法和刷表法
- ☑ 掌握记忆化搜索在实现方面的注意事项
- ☑ 掌握记忆化搜索和递推中输出方案的方法
- ☑ 掌握递推中滚动数组的使用方法
- ☑ 熟练解决经典动态规划问题

动态规划的理论性和实践性都比较强，一方面需要理解“状态”、“状态转移”、“最优子结构”、“重叠子问题”等概念，另一方面又需要根据题目的条件灵活设计算法。可以说，对动态规划的掌握情况在很大程度上能直接影响一个选手的分析和建模能力。

9.1 数字三角形

动态规划是一种用途很广的问题求解方法，它本身并不是一个特定的算法，而是一种思想，一种手段。下面通过一个题目阐述动态规划的基本思路和特点。

9.1.1 问题描述与状态定义

数字三角形问题。有一个由非负整数组成的三角形，第一行只有一个数，除了最下行之外每个数的左下方和右下方各有一个数，如图 9-1 所示。

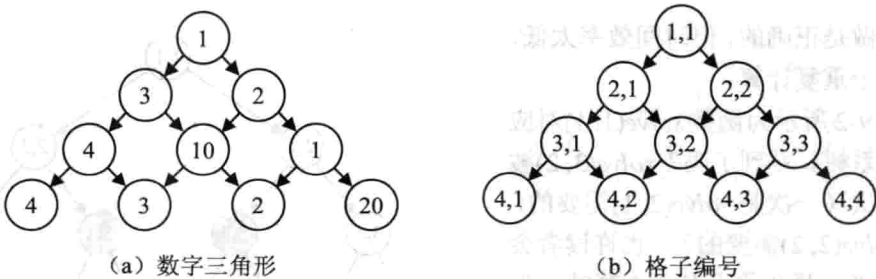


图 9-1 数字三角形问题

从第一行的数开始，每次可以往左下或右下走一格，直到走到最下行，把沿途经过的

数全部加起来。如何走才能使得这个和尽量大？

【分析】

如果熟悉回溯法，可能会立刻发现这是一个动态的决策问题：每次有两种选择——左下或右下。如果用回溯法求出所有可能的路线，就可以从中选出最优路线。但和往常一样，回溯法的效率太低：一个 n 层数字三角形的完整路线有 2^{n-1} 条，当 n 很大时回溯法的速度将让人无法忍受。

为了得到高效的算法，需要用抽象的方法思考问题：把当前的位置 (i, j) 看成一个状态（还记得吗？），然后定义状态 (i, j) 的指标函数 $d(i, j)$ 为从格子 (i, j) 出发时能得到的最大和（包括格子 (i, j) 本身的值）。在这个状态定义下，原问题的解是 $d(1, 1)$ 。

下面看看不同状态之间是如何转移的。从格子 (i, j) 出发有两种决策。如果往左走，则走到 $(i+1, j)$ 后需要求“从 $(i+1, j)$ 出发后能得到的最大和”这一问题，即 $d(i+1, j)$ 。类似地，往右走之后需要求解 $d(i+1, j+1)$ 。由于可以在这两个决策中自由选择，所以应选择 $d(i+1, j)$ 和 $d(i+1, j+1)$ 中较大的一个。换句话说，得到了所谓的状态转移方程：

$$d(i, j) = a(i, j) + \max\{d(i+1, j), d(i+1, j+1)\}$$

如果往左走，那么最好情况等于 (i, j) 格子中的值 $a(i, j)$ 与“从 $(i+1, j)$ 出发的最大总和”之和，此时需注意这里的“最大”二字。如果连“从 $(i+1, j)$ 出发走到底部”这部分的和都不是最大的，加上 $a(i, j)$ 之后肯定也不是最大的。这个性质称为最优子结构（optimal substructure），也可以描述成“全局最优解包含局部最优解”。不管怎样，状态和状态转移方程一起完整地描述了具体的算法。

提示 9-1：动态规划的核心是状态和状态转移方程。

9.1.2 记忆化搜索与递推

有了状态转移方程之后，应怎样计算呢？

方法 1：递归计算。程序如下（需注意边界处理）：

```
int solve(int i, int j){
    return a[i][j] + (i == n ? 0 : max(solve(i+1, j), solve(i+1, j+1)));
}
```

这样做是正确的，但时间效率太低，其原因在于重复计算。

如图 9-2 所示为函数 $\text{solve}(1, 1)$ 对应的调用关系树。看到了吗？ $\text{solve}(3, 2)$ 被计算了两次（一次是 $\text{solve}(2, 1)$ 需要的，一次是 $\text{solve}(2, 2)$ 需要的）。也许读者会认为重复算一两个数没有太大影响，但事实是：这样的重复不是单个结点，而是一棵子树。如果原来的三角形有 n 层，则调用关系树也会有 n 层，一共有 $2^n - 1$

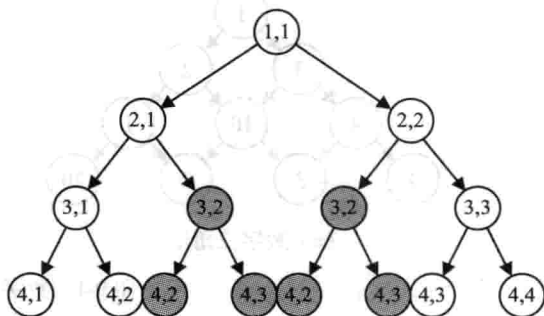


图 9-2 重叠子问题

个结点。

提示 9-2: 用直接递归的方法计算状态转移方程，效率往往十分低下。其原因是相同的子问题被重复计算了多次。

方法 2: 递推计算。程序如下（需再次注意边界处理）：

```
int i, j;
for(j = 1; j <= n; j++) d[n][j] = a[n][j];
for(i = n-1; i >= 1; i--)
    for(j = 1; j <= i; j++)
        d[i][j] = a[i][j] + max(d[i+1][j], d[i+1][j+1]);
```

程序的时间复杂度显然是 $O(n^2)$ ，但为什么可以这样计算呢？原因在于： i 是逆序枚举的，因此在计算 $d[i][j]$ 前，它所需要的 $d[i+1][j]$ 和 $d[i+1][j+1]$ 一定已经计算出来了。

提示 9-3: 可以用递推法计算状态转移方程。递推的关键是边界和计算顺序。在多数情况下，递推法的时间复杂度是：状态总数 $\times$ 每个状态的决策个数 $\times$ 决策时间。如果不同状态的决策个数不同，需具体问题具体分析。

方法 3: 记忆化搜索。程序分成两部分。首先用 “`memset(d, -1, sizeof(d));`” 把 d 全部初始化为 -1，然后编写递归函数^①：

```
int solve(int i, int j){
    if(d[i][j] >= 0) return d[i][j];
    return d[i][j] = a[i][j] + (i == n ? 0 : max(solve(i+1, j), solve(i+1, j+1)));
}
```

上述程序依然是递归的，但同时也把计算结果保存在数组 d 中。题目中说各个数都是非负的，因此如果已经计算过某个 $d[i][j]$ ，则它应是非负的。这样，只需把所有 d 初始化为 -1，即可通过判断是否 $d[i][j] \geq 0$ 得知它是否已经被计算过。

最后，千万不要忘记在计算之后把它保存在 $d[i][j]$ 中。根据 C 语言“赋值语句本身有返回值”的规定，可以把保存 $d[i][j]$ 的工作合并到函数的返回语句中。

上述程序的方法称为记忆化 (memoization)，它虽然不像递推法那样显式地指明了计算顺序，但仍然可以保证每个结点只访问一次，如图 9-3 所示。

由于 i 和 j 都在 $1 \sim n$ 之间，所有不相同的结点一共只有 $O(n^2)$ 个。无论以怎样的顺序访问，时

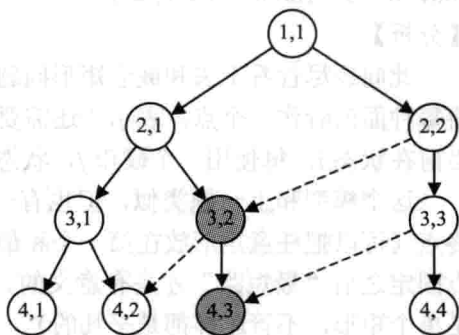


图 9-3 记忆化搜索

^① 注意这个函数的工作方式并不像它表面显示的那样——如果把 -1 改成 -2，并不是在把所有 d 值都初始化为 -2！请只用 0 和 -1 作为“批量赋值”的参数。